BitFlip

CloudSherpa

Testing Policy Document

Contents

1 Introduction

1.1 Purpose and Objectives

The purpose of Team Bitflip's testing policy is to accelerate software development by establishing a fast and reliable feedback system that identifies defects before code is deployed. Testing is treated as an integral part of the development lifecycle rather than a final verification step. By writing tests throughout development, developers are encouraged to produce loosely coupled, modular, and maintainable code with well-defined abstractions and dependencies. This results in software that is easier to extend, refactor, and validate over time.

For the CloudSherpa project, comprehensive testing is particularly critical due to the platform's responsibility of collecting, processing, and analysing monitoring data from multiple cloud providers. Failures in cloud integrations, metric collection, or data processing can result in inaccurate analytics, incorrect recommendations, or unintended provider billing. A robust testing strategy therefore ensures the correctness, reliability, and consistency of data across AWS, Google Cloud Platform, and Microsoft Azure integrations while allowing new features to be confidently introduced.

Testing is fully integrated into the team's Continuous Integration workflow. Every code change is validated before it can be merged into the development or main branch, ensuring that only verified, production-ready code enters the shared codebase.

The primary objectives of this testing policy are to:

- **Ensure software reliability:** Detect defects early and verify that new functionality behaves as expected without introducing regressions.
- **Protect data integrity:** Validate that CloudSherpa accurately retrieves, processes, and stores monitoring data across multiple cloud providers.
- **Promote maintainable software design:** Encourage modular architectures, loose coupling, and high testability by requiring code to be easily unit tested.
- **Enforce quality through CI/CD:** Require automated tests to pass before code can be merged, ensuring a stable and consistently deployable codebase.
- **Reduce project risk:** Minimise production defects, improve deployment confidence, and shorten debugging and maintenance efforts through comprehensive automated testing.

2 Roles and Responsibilities

Quality assurance is a shared responsibility across the development team. By clearly defining testing roles and responsibilities, Team Bitflip promotes technical ownership throughout every stage of the software development lifecycle while ensuring that quality is maintained through collaborative effort.

Quality Assurance (QA) Lead / Tester

- **Test review and validation:** Review the unit tests submitted with new features to ensure they provide sufficient coverage, adequately test edge cases and error conditions, and meet the team's quality standards before the code is merged into the shared codebase.
- **Integration and end-to-end testing:** Design, implement, and maintain integration and end-to-end tests that validate interactions between system components, cloud provider integrations, and critical user workflows. Review and validate tests contributed by other testers to ensure consistency and completeness.
- **Test coverage management:** Monitor testing coverage across services and application layers, identify gaps in the existing test suite, and recommend or develop additional tests to ensure that critical functionality is thoroughly validated.
- **Regression testing:** Coordinate regression testing for significant feature additions, bug fixes, and release candidates to ensure that previously implemented functionality continues to operate correctly.
- **Quality reporting:** Track testing results, report defects with sufficient detail for reproduction, and communicate testing progress and outstanding quality risks to the development team prior to release.

Software Engineer / Developer

- **Unit testing:** Each developer is responsible for writing and maintaining comprehensive unit tests for the features they implement. Tests should cover expected behaviour, edge cases, and error conditions to ensure the correctness and reliability of the code.
- **Code quality and pull requests:** Before submitting a pull request, developers must ensure that all existing and newly added tests pass locally

and within the Continuous Integration (CI) pipeline. Pull requests should include sufficient test coverage for any new or modified functionality.
- **Code reviews:** Developers reviewing pull requests are responsible for verifying that implementations meet the project's quality standards, that adequate tests have been included, and that no regressions or untested edge cases have been introduced.
- **Bug reporting and resolution:** All team members are responsible for reporting defects as soon as they are discovered. Developers should investigate reported issues, implement appropriate fixes, and add regression tests to prevent similar defects from recurring.
- **System validation:** Developers actively participate in testing the application as a whole, including integration testing where appropriate, to verify that interactions between system components and cloud providers function correctly.

Product Owner / Project Manager

- **Backlog management:** Define and maintain the product backlog, ensuring that user stories accurately reflect business requirements and project priorities.
- Specify clear and measurable acceptance criteria for every user story to guide development and testing activities.
- Acceptance criteria: Approve or reject completed user stories based on whether they satisfy the functional and business requirements.
- **Defect prioritization:** Prioritise defects according to their business impact and determine whether issues must be resolved before release.
- **Vision alignment:** Collaborate with developers and testers to clarify requirements, resolve ambiguities, and ensure that delivered features align with stakeholder expectations.
- **Project approval:** Provide final approval for production releases from a business perspective once all quality gates and acceptance criteria have been satisfied.

3 Testing Types and Phases

Testing is performed at multiple levels, with each level targeting a different aspect of the system to identify defects as early as possible. Clear responsibilities, testing standards, and quality metrics are established for each testing phase to ensure consistent validation of new functionality before it is integrated into the shared codebase.

</> 1. Unit Testing

- **Scope:** Unit tests verify core functionality within the project, including backend logic, interactive frontend features, and UI components.
- **Responsibility:** Unit tests are written by the same developers that were responsible for the feature, ensuring that they are familiar with the code that they are testing and can make changes to the code if any issues are identified during the testing process.
- **Standard:** Each service requires a testing coverage of 80% as verified on SonarQube. All edge cases must be tested for and all tests must be passing before a merge to the shared codebase.

2. Integration Testing

- **Scope:** Integration tests test interactions between modules such as two backend modules that are associated, backend API endpoints interacting with the database, as well as frontend state management interacting with backend routes.

  Examples of these cases include our Service interacting with backend API endpoints, our Ingestion service sending write requests to the database, or our frontend state changing after interactions with the Service.
- **Responsibility:** Unit tests are written primarily by our testers, who are responsible for writing integration tests that cover all interaction cases between modules, following a bottom-up testing approach where drivers are used for untested modules.
- **Standard:** All important interactions between modules must be tested, with focus placed on modules that are used most often, as well as modules where interaction mismanagement may lead to negative outcomes. An example of this includes our scheduler sending duplicate requests to our ingestion service, which would cause CloudSherpa customers to get billed on our free service tier.

3. End-to-End (E2E) Testing

- **Scope:** End-to-end tests verify full user stories, starting at frontend user interaction, moving to request processing and response handling, and then verifying the final result as seen by the user.

  An example of this would be registration. A user should be able to enter the appropriate information to be able to register, which should create a database entry, provide the user with feedback, and redirect the user to the appropriate webpage.

- **Responsibility:** End-to-end tests are performed collaboratively by the entire development team. All team members work together to validate complete user workflows, ensuring that all system components function correctly when integrated and that features meet both technical and user requirements before release.

- **Standard:** All critical user journeys must be successfully verified before a feature is considered complete. These tests must confirm that user interactions produce the expected behaviour across the frontend, backend, database, and any external services.

  Particular emphasis is placed on high-impact workflows such as user authentication, account management, cloud account onboarding, and data ingestion, as failures in these areas would have the greatest impact on the user experience and system reliability.

4 Tools and Environments

When discussing the tools and environments used for testing it is worth considering the different technologies and frameworks used in the development of CloudSherpa since the tools were chosen and environments set up accordingly.

4.1 Spring Boot Applications

The spring-boot-starter-test contains a bundle of tools required for testing Spring Boot Applications, from this bundle we use

- **JUnit5** - A Java framework for writing and running unit and integration tests
- **Spring Test & Spring Boot Test** - Core utilities supporting Spring context environments for testing
- **Mockito** - Framework used for mocking component dependencies and verifying isolated behaviour for unit tests
- **AssertJ** - A library that contains assertion utilities

Additionally, the project uses **Testcontainers** to support integration testing. Testcontainers creates temporary containerised instances of the database for each test suite. Because the production database is also containerised, these tests closely resemble the production environment. The disposable nature of the containers also ensures that each test suite runs in a clean and consistent environment, preventing side effects from one suite from propogating to subsequent suites.

4.2 Next.js Application

For the Next.js dashboard application, the project uses the **Vitest** framework for unit and component testing. Vitest provides testing support for the TypeScript, JSX, and React technologies used by the application.

4.3 End to End Tests

We use the **Playwright** web automation framework for End-to-End testing accross multiple browsers and platforms.

4.4 Continuous Integration

The continuous integration pipelines automatically run unit, integration, and end-to-end tests. These test suites act as regression tests and deployment gates.

Unit and integration tests are executed for every pull request targeting the `dev` or `main` branch. End-to-end tests are executed for pull requests targeting the `staging` or `main` branch, allowing the fully built system to be validated before deployment.

5 Defect Management Process

Defects identified during development, testing, or user acceptance testing are logged on GitHub isues and assigned an appropriate severity level based on their impact on the system and its users. Each defect is reproduced, investigated, and assigned to the developer responsible for the affected component.

Once a fix has been implemented, the patch is verified by a tester before the issue is marked as resolved and merged into the shared codebase. If a defect is found after deployment, it is prioritised according to its severity and addressed in the next appropriate release or, where necessary, through an immediate hotfix.

5.1 Defect Severity Levels

- **HIGH**: Critical defects that prevent normal operation of the system or pose a significant risk to users. Examples include application crashes, data loss, incorrect billing calculations, security vulnerabilities, authentication failures, or failures in cloud resource ingestion. These defects must be addressed immediately and fixed before any new features are developed or released.
- **MEDIUM**: Defects that significantly impact functionality but have an acceptable workaround or affect only a subset of users. Examples include broken application features, incorrect data presentation, degraded performance, or failures affecting non-critical workflows. These defects should be resolved within the current development iteration before the next planned release.
- **LOW**: Minor defects that have little or no impact on core functionality, such as visual inconsistencies, spelling or grammatical errors, minor usability issues, or cosmetic layout problems. These defects are prioritised after higher-severity issues and are addressed as time permits during normal development.

5.2 Logging a Defect

Bugs are logged on GitHub issues, with the following report template to ensure reproducability as well as to ensure that the defect can be properly handled such that the expected behaviour can be achieved.

Standard Bug Report Template

Title: [Brief, descriptive summary of the issue]
Severity: [High / Medium / Low]
Environment: [Dev / Staging / Prod] — **Browser:** [Chrome/Firefox/Edge]

Steps to Reproduce:

1. *[Step 1 description]*
2. *[Step 2 description]*
3. *[Step 3 description]*

Expected Result: *[What should the system do?]*
Actual Result: *[What did the system actually do?]*
Attachments: [Insert screenshots, console errors, or server logs here]

6 Acceptance Criteria & Definition of Done

A feature is considered complete only once it has satisfied the team's quality, testing, and review requirements. The following checklist encapsulates requirements that, once achieved, show that all completed work is functional, maintainable, and ready for deployment to production.

Release Criteria Checklist (Definition of Done)

A feature is considered "Done" and ready for production when:

- [x] The implementation satisfies all functional requirements defined in the associated user story or task.
- [x] The code has been reviewed and approved by at least two other team members.
- [x] All unit, integration, and end-to-end tests have been completed successfully, with no failing test cases.
- [x] New code achieves the required minimum unit test coverage of 80% as verified by SonarQube.
- [x] All identified defects of High and Medium severity have been resolved, with no critical issues remaining.
- [x] The feature has been manually verified to function correctly within the complete application.
- [x] Any required documentation, configuration files, or API documentation have been updated to reflect the implemented changes.
- [x] The feature has been demonstrated to the project owners or stakeholders and has received their approval.
- [x] The feature has been successfully merged into the shared codebase and passes all CI/CD pipeline checks without errors.